# State Space Search

- State Space Search is a **problem-solving approach** in Artificial Intelligence (AI) in which all possible states of a system are explored to find a solution.
- This method is used in various problems like **puzzle solving, route planning, and AI decision-making**.

# Components of state space search

- **State:** A situation or condition in the problem. A specific configuration of the problem.
- **Initial State:** The starting point of the search.
- **Goal State:** The desired solution.
- **Transition: (Actions):** Steps that move from one state to another.
- **Path:** A sequence of states connected by transitions.
- **State Space:** The collection of all possible states.

# Travelling Salesman Problem (TSP)

- The Travelling Salesman Problem (TSP) is a well-known optimization problem in computer science and operations research.
- The problem is defined as follows:
  - given a set of cities and the distances between them,
  - find the shortest possible route that visits each city exactly once and returns to the starting city.
  - With minimum travelling cost.
- TSP can be classified into **Symmetric TSP (STSP)** and **Asymmetric TSP (ATSP)** based on the nature of the cost between cities.

# State Space Representation in TSP

- **State:** A city and the set of cities visited so far.
- **Initial State:** The salesperson starts from a given city.
- **Operators (Actions):** Moving to a **new, unvisited city**.
- **State Space:** All possible **permutations of city visits**.
- **Goal State:** All cities are visited exactly once, and the salesperson returns to the starting city.

# 1. Symmetric TSP (STSP)

•The distance or cost between two cities is the same in both directions.

•**Mathematically:** If there is a path between city Aand city B, then

$$d(A,B)=d(B,A)$$

•**Representation:** The cost matrix is symmetric, meaning C[i][j]=C[j][i]

•**Example:** Road networks where distances between cities are the same regardless of the direction.

| | **A** | **B** | **C** | **D** |
|---|---|---|---|---|
| **A** | 0 | 10 | 15 | 20 |
| **B** | 10 | 0 | 35 | 25 |
| **C** | 15 | 35 | 0 | 30 |
| **D** | 20 | 25 | 30 | 0 |

## 2. Asymmetric TSP (ATSP)

•The distance or cost between two cities differs depending on the direction.
•**Mathematically:** d(A,B)≠d(B,A)
•**Representation:** The cost matrix is asymmetric, meaning C[i][j]≠C[j][i]C[i][j]
•**Example:**

- One-way streets in cities
- Flight costs (which vary depending on direction due to factors like wind or layovers)

| | **A** | **B** | **C** | **D** |
|---|---|---|---|---|
| **A** | 0 | 12 | 15 | 22 |
| **B** | 8 | 0 | 30 | 25 |
| **C** | 14 | 35 | 0 | 28 |
| **D** | 18 | 24 | 30 | 0 |

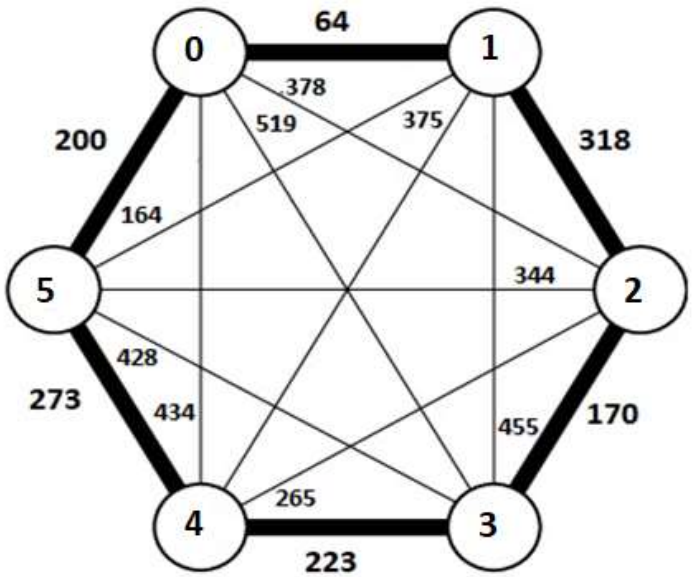
# Brute force approach

- A brute force approach is an approach that finds all the possible solutions to find a adequate solution to a given problem. The brute force algorithm tries out all the possibilities till a adequate solution is not found.
- In this approach, we'll generate all the possible permutations (routes). And find the one with the shortest distance.

The algorithm is as follows:

- **Step 1:** Choose a city to act as a starting point(let's say city1).
- **Step 2:** Find all possible routes using permutations. For n cities, it will be (n-1)!.
- **Step 3:**Calculate the cost of each route and return the one with the minimum cost.

# I. Write a program for a 6-city symmetric TSP using a brute-force approach.




| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | **0** | **64** | **378** | **519** | **434** | **200** |
| 1 | **64** | **0** | **318** | **455** | **375** | **164** |
| 2 | **378** | **318** | **0** | **170** | **265** | **344** |
| 3 | **519** | **455** | **170** | **0** | **223** | **428** |
| 4 | **434** | **375** | **265** | **223** | **0** | **273** |
| 5 | **200** | **164** | **344** | **428** | **273** | **0** |

# TSP using Brute Force - Example

| | P | Q | R | S |
|---|---|---|---|---|
| **P** | 0 | 3 | 6 | 8 |
| **Q** | 3 | 0 | 9 | 4 |
| **R** | 6 | 9 | 0 | 2 |
| **S** | 8 | 4 | 2 | 0 |

**Possible Paths & Costs:**

- P→Q→R→S→P= **22**
- P→Q→S→R→P= **15**
- P→R→Q→S→P= **27**
- P→R→S→Q→P = **15**
- P→S→R→Q→P = **27**
- P→S→Q→R→P = **22**

**Optimal Paths:**

- P→Q→S→R→P
- P→R→S→Q→P
- **Both have a cost of 15**.

# TSP using Brute Force Analysis

- For Different Values of n:

➢ n=2:

- 1 Route :
  - a→b→a
- (n−1)! = 1! = 1

➢ n=3:

- 2 Routes:
  - a→b→c→a
  - a→c→b→a
- (3−1)! = 2! = 2

➢ n=4:

- 6 Routes:
  - a→b→c→d→a
  - a→b→d→c→a
  - a→c→b→d→a
  - a→c→d→b→a
  - a→d→b→c→a
  - a→d→c→b→a
- (4−1)! = 3! = 6

# Nearest Neighbour Algorithm - TSP

| | A | B | C | D |
|---|---|---|---|---|
| **A** | 0 | 1 | 6 | 3 |
| **B** | 1 | 0 | 3 | 2 |
| **C** | 6 | 3 | 0 | 1 |
| **D** | 3 | 2 | 1 | 0 |

- **Solution Steps:**

1. **Start with vertex A**.
2. **Nearest neighbor** →B,C,D
   - Choose **(A, B) = 1**.
1. **From B**, Nearest neighbor →C,D
   Choose **(B, D) = 2**.
2. **From D**, Nearest neighbor → C
   Choose **(D, C) = 1**.
3. **From C**, return back to **A (C, A) = 6**.
4. **A→B→D→C→A = 10**

**Total Cost Calculation:**

- (A−B)=1
- (B−D)=2
- (D−C)=1
- (C−A)=6
- **Total Cost** = **10**

**Optimal Tour:**

- A→B→C→D→A =8
  - (A−B)=1
  - (B−C)=3
  - (C−D)=1
  - (D−A)=3
  - **Total cost = 8**

**Approximation ratio= 10/8 = 1.25**